Redis Persistence Risk & Mitigation

Instance: `physioguidance-redis` (Memorystore, me-central2) **Tier:** BASIC (single node, no replica) **Size:** 1 GiB **Version:** Redis 7.0 **Persistence: DISABLED** (no RDB snapshots, no AOF log) **Verified:** 2026-08-17 (`gcloud redis instances describe`)

1. What "no persistence" actually means

Every key in Redis lives only in RAM. If the instance restarts — planned maintenance, node failure, OOM eviction, GCP-side patch — **all keys are lost**.

There is no snapshot to reload from and no append-only log to replay. On restart the instance comes back **empty**, and application code sees `MISS` for every prior key until traffic re-populates them.

2. What we store in Redis

Data	Owner	Loss impact
BullMQ job queues — `webhook-processing` , `wcag` , `xapi-statements`	`payments` , `processing` , `xapi` modules	**HIGH** — in-flight jobs are dropped. See §3 per-queue impact.
Distributed locks (`DistributedLockService`) — cron coordination between api-web replicas	`tasks` module	**LOW** — locks re-acquire on next @Cron tick (worst case: one cron fires twice in the restart window).
Rate limit counters (`@nestjs/throttler`)	Framework	**LOW** — counters reset, briefly permitting more requests, then re-enforce. Not a security regression.
Session/JWT state	—	**NONE** — JWTs are stateless; refresh tokens live in Postgres.
User-facing cache	—	**NONE** — we do not use Redis as an app cache; all reads hit Postgres.

Verified via: `grep -rn "BullModule.registerQueue\|@InjectQueue" backends/src` (2026-08-17).

3. Queue-by-queue loss impact

3.1 `webhook-processing` (payments)

- **Producer:** Moyasar / Tamara / Tabby / Bank Transfer webhooks land at `POST /webhooks/*` , enqueue for verification + reconciliation.

- **Loss impact:** A dropped webhook means one payment status is stuck in `pending`. Not silent — reconciliation cron (`reconciliation-cron.controller.ts`) reconciles against the gateway API every 5 min, so the delay is bounded.
- **User-visible effect:** Student sees "processing" for up to ~5 min longer than usual on the receipt page.
- **Data loss:** None. Gateway is the source of truth; we re-fetch on next reconciliation.

3.2 `wcag` (processing)

- **Producer:** After a Zoom recording transcript arrives, `TranscriptProcessor` enqueues a WCAG accessibility check.
- **Loss impact:** A dropped job means one recording's WCAG compliance flag is not set until the recording is re-processed.
- **User-visible effect:** None immediately. Admin sees "pending" in NELC compliance dashboard for that recording.
- **Recovery:** Re-run the transcript job manually from admin dashboard, or wait for the next `RecoverStuckRecordings` cron.

3.3 `xapi-statements` (xapi)

- **Producer:** Learning-record events (course-progress, quiz-attempt) enqueued for xAPI LRS delivery.
- **Loss impact:** Some statements never reach the LRS. Since xAPI is a compliance/analytics stream (not user-facing), the effect is a small gap in learning-analytics reports for the outage window.
- **Recovery:** No auto-replay currently. Documented as an accepted gap — see §5 mitigations.

---

4. Failure modes we've considered

Mode	Frequency	Recovery
GCP-scheduled Memorystore maintenance	Rare (~quarterly) — GCP notifies 7 days ahead via email	Cache repopulates from traffic; queues resume.
Node crash / OOM	Rare — we run at ~15% of 1 GiB	Same as above. Restart typically <60 seconds.
Region outage (me-central2)	Very rare	See regional-failure-playbook.md — Redis is regional; a full-region outage requires cross-region recovery, not just Redis restart.
OOM eviction under load spike	Possible if we bump to BullMQ-heavy features	Non-issue today (see §6 monitoring).

5. Mitigations in place

1. **Reconciliation cron for payments** — every 5 min, re-fetches gateway status for any `pending` webhook, so a dropped webhook cannot silently strand a payment.
2. `RecoverStuckRecordings` **cron** — hourly, re-enqueues WCAG jobs for recordings marked processing >1h.
3. **DistributedLockService is idempotent** — a lost lock only causes at-most-one duplicate cron execution; every @Cron method must (and does) tolerate double-execution.
4. **JWT stateless auth** — a Redis restart never logs users out.
5. **Postgres is the source of truth** — all persistent state (users, courses, payments, certificates, enrollments) is in Cloud SQL, which has PITR + daily automated backups + deletion protection.

6. What is deliberately NOT done (and why)

6.1 STANDARD_HA tier (replica for HA)

- **Cost:** ~$34/mo delta (1 GiB BASIC → 1 GiB STANDARD_HA).
- **Benefit:** Automatic failover on primary node crash — restart drops from ~60s to <10s. Data is still not persisted across a full-region incident.
- **Decision:** Not upgraded. The user-visible impact of a 60s Redis outage (some webhooks delayed by ≤5 min via reconciliation, some xAPI events lost) does not justify the cost at current traffic. **Revisit** when we hit ≥1000 daily paid transactions or when a Redis-restart incident causes measurable harm.

6.2 RDB / AOF persistence

- **Available on:** Memorystore Redis STANDARD_HA only (not BASIC).
- **Would prevent:** Queue loss on restart.
- **Decision:** Blocked by tier choice above. Reconsider together with §6.1.

6.3 xAPI statement replay

- **Would prevent:** Analytics gaps on Redis loss.
- **Cost to build:** ~1 day (outbox pattern: write statement to Postgres first, worker drains to LRS, delete row on success).
- **Decision:** Deferred — xAPI is compliance/analytics, not user-facing, and gap tolerance is acceptable for the current audit scope. **Revisit** if NELC or Amna requires zero-loss learning analytics.

7. Monitoring & alerting

- **Memorystore memory usage** — visible in the admin **Monitoring** dashboard (Resources → Redis) once Amna grants `monitoring.viewer` to `api-runtime@`.
- **BullMQ job failures** — logged to Cloud Logging with severity `ERROR`, visible in `/dashboard/admin/logs` and in Cloud Console → Logs Explorer.
- **No paging alert on Redis restart today.** If desired, add a Cloud Monitoring alert on `redis.googleapis.com/stats/uptime` reset — ~5 min config work, not currently in scope.

---

8. Runbook — Redis restart / cache loss

Detection:

- Monitoring dashboard shows Redis metrics reset to zero, OR
- Cloud Logging shows a burst of `ioredis` reconnection messages, OR
- Backend `/health/ready` briefly returns `redis: unavailable` before reconnecting.

Response (typically no action needed):

1. Confirm Redis is back up: `gcloud redis instances describe physioguidance-redis --region=me-central2 --format="value(state)"` should return `READY`.
2. Check payment reconciliation ran within the last 5 min: `/dashboard/admin/finance` → recent webhooks — any stuck payments will clear on the next reconciliation tick.
3. If a WCAG-flagged recording is stuck >1h, manually re-enqueue from `/dashboard/admin/compliance`.
4. For xAPI: accept the gap — document in the compliance log if audit requires.

Escalation: Only if Redis stays down >10 min or reconciliation cron itself is failing — then the incident is broader than "Redis restart" and requires investigation of the api-web service.

---

9. Summary for Amna

- Redis holds **no user data**. It holds queue jobs and rate-limit counters.
- On restart, some in-flight jobs are lost — but every consequential job (payments, WCAG) has a **catch-up mechanism** in Postgres cron.
- The choice to run BASIC-tier without persistence is a **~$34/mo cost decision**. When traffic grows or when the loss window matters, upgrade to STANDARD_HA and enable persistence — the change is

a `gcloud redis instances update --tier=STANDARD_HA` command plus a ~2 min instance restart.

- No urgent action required today.